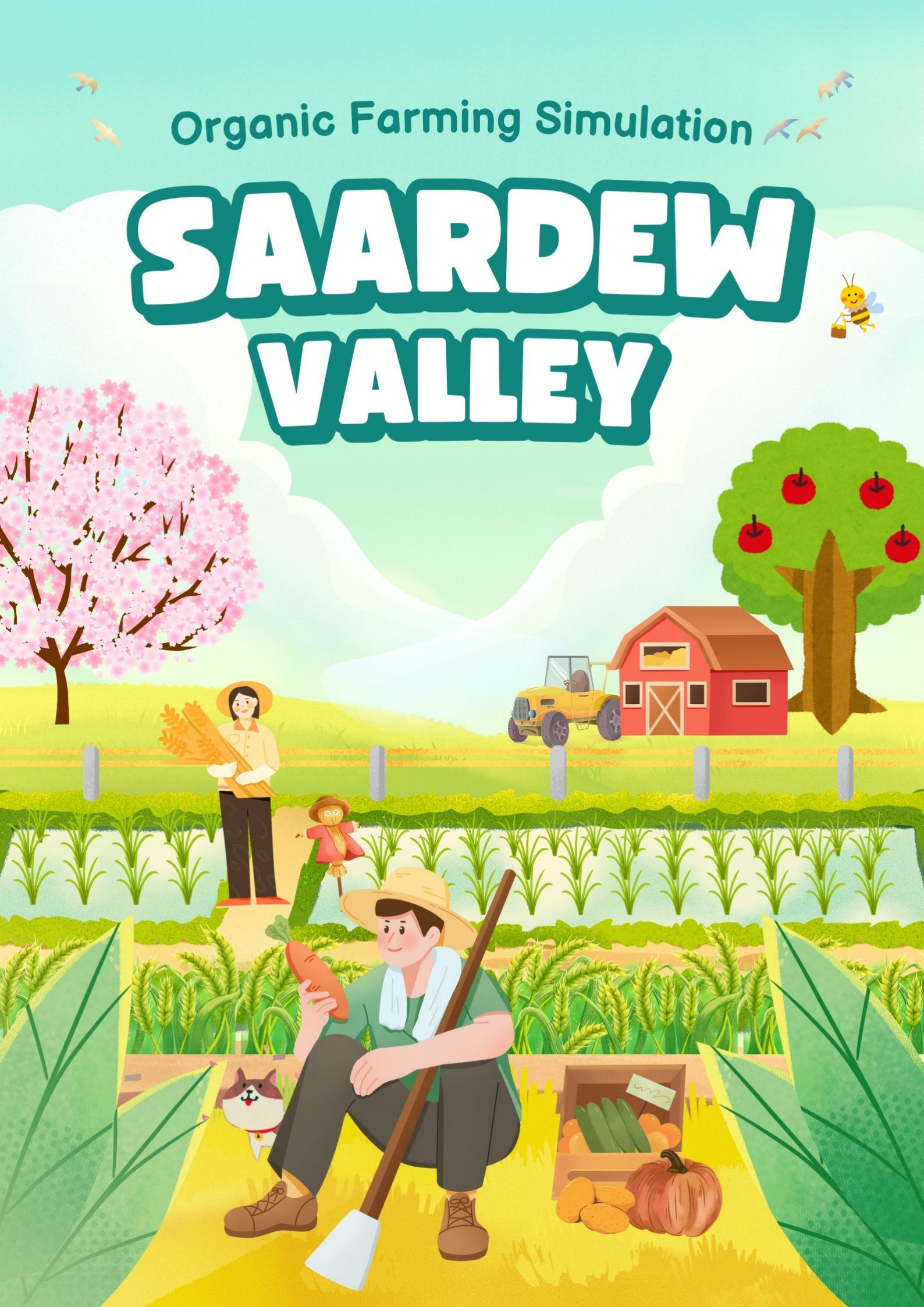
Organic Farming Simulation

SAARDEW VALLEY



Contents

1. Organizational Matters

Since you have already completed the exercise phase, the group phase of the Software Engineering Lab begins now. This group phase is divided into a design phase (∼2–2.5 weeks) and a subsequent implementation phase (∼2–3 weeks).

In this group phase, you will first design, and then implement and test a simulator together with your fellow students in a group. Group assignment will be done by the chair on the first day of the group phase. You will be assigned a group in the CMS. To work as a group on the project, we will provide your group access to a workroom at the university for the duration of the group phase. As an additional service, a tutor will be available to assist you and will be in regular contact with your group. Your tutor will let you know which room is available for you. You can also use the rooms outside of the core hours in accordance with your tutor.

For the design phase, you will spend the first two weeks (approximately) of the group phase designing your software system to match the specified simulation. In a design review, you will present your designs to a member of the Chair of Software Engineering and receive feedback. Like customers in real life, the chair is volatile in its requirements for the simulator to be implemented. Therefore, there will be a change of the simulation setting after the design review. Your design must be kept flexible enough to accommodate the changes in the task without much effort. The specific changes will then be announced in a separate document after the design reviews. In the final days of the design phase, there will be a design defense, in which your group's design and your individual understanding of the design are evaluated. In case your group does not pass this evaluation, you will get a chance to rework the group's design and present it in another defense a few days later during the design re-evaluation. In case you individually do not pass this evaluation, you will get a chance to prove your understanding of the design in an exam during the design re-evaluation. You can only continue the SE Lab if you have passed the design phase both as a group and as an individual.

After the design re-evaluation, the implementation phase follows with a duration of (approximately) two weeks. During this time, your group must implement and test the project. The implementation includes the parsing of the simulation scenario as well as the execution of the simulation logic. In addition, you will need to develop a comprehensive test suite, including unit tests for individual components, integration tests for interactions between components, and system tests for the entire simulation. Tests should cover both standard and special cases that may arise. You will present and receive feedback on your implementation in the code review, and submit your final implementation including your test suite for evaluation.

You can only pass the implementation phase if your group's implementation has passed our evaluation and your contribution was sufficient, or you have passed the subsequent individual oral re-evaluation, in which you need to defend your existing contribution.

Please note:

For the schedule of the group phase, please additionally note the information from the document on Organizational Matters, which has already been provided to you in the CMS at the beginning of the SE Lab.

1.1. Design

Regarding the design, your group must complete the following tasks:

1. You must create a UML class diagram of your system. The class diagram must contain all relevant classes of your implementation, as well as their most important properties and methods. It should show which data and which functionality you encapsulate in each class. The relations of the classes among themselves must be modeled correctly. In the class diagram, all classes as well as all non-trivial methods together with their parameters must be specified. Trivial methods include, in particular, `hashCode`, `equals`, and `toString`.
2. You need to model the simulation in a state diagram to better understand the simulation flow and to ensure that you also fully consider this simulation flow in your design. Create a state diagram that begins at the start of the simulation (i.e., the configuration files are not yet loaded) and models the simulation itself up to the end of the simulation. Make the diagram as fine-granular and detailed as possible.
3. You need to demonstrate the interaction of your classes using UML sequence diagrams for the following scenarios:
   - Initialization of the simulation, with ($\texttt{max\_ticks} = 0$), until the program terminates.
   - Handling a single simulation tick, from the start of tick 32 ($\texttt{yeartick} = 19$) until the next tick starts. You can find the map of the simulation in Figure 9. The soil moisture is $1000\,\texttt{L}$ for all tiles except for tiles $\texttt{id} = 42, 46, 50, 54$, where the soil moisture is $600\,\texttt{L}$ before the beginning of the tick. There is one farm ($\texttt{id} = 34$) with three machines on the map, all located on a shed at tile 12. One machine ($\texttt{id} = 1$) can perform `SOWING` and `IRRIGATING` for the plants `WHEAT` and `PUMPKIN` with $\texttt{duration} = 4$. The second machine ($\texttt{id} = 2$) can perform `HARVESTING` for the plants `OAT` and `PUMPKIN` with $\texttt{duration} = 10$. The third machine ($\texttt{id} = 3$) can perform `WEEDING` and `IRRIGATING` for the plant `PUMPKIN`

with `duration` $= 4$. For tick 32, a sowing plan with `id` $= 3$ orders the sowing of `WHEAT` at tiles `id` $= 40, 43, 44, 48, 52$. The tile 43 does contain only `POTATO` as `possiblePlant` and the tile 48 was fallow for only 1 tick. The harvest estimate on the tiles $42, 46, 50, 54$ is $320'850\,\mathrm{g}$[1] at the beginning of the tick. There are no incidents and no environmental influences on the harvest.

- Handling a single simulation tick, from the start of tick 44 (`yeartick` $= 8$) until the next tick starts. You can find the map of the simulation in Figure 10. The soil moisture is $1000\,\mathrm{L}$ for all tiles except for tiles `id` $= 150, 17, 18$, where the soil moisture is $200\,\mathrm{L}$ before the beginning of the tick. The harvest estimate is $20'000\,\mathrm{g}$ for tile 150, $30'000\,\mathrm{g}$ for tile 17, and $40'000\,\mathrm{g}$ for tile 18. There are 3 clouds on the map, cloud 3 with $4'000\,\mathrm{L}$ at tile 23, cloud 5 with $2'000\,\mathrm{L}$ at tile 18, and cloud 9 with $4'500\,\mathrm{L}$ at tile 12. The tiles with `id` 17 and 18 have a maximum of soil moisture of $1'000$ and a current value of 200 before the start of the tick. There exists one farm (`id` $= 2$) with a machine (`id` $= 4$) on tile 30 whose action is `HARVESTING`. Two incidents must be performed this tick, an `ANIMAL_ATTACK` incident (`id` $= 2$) with location 19 and radius 2, then a `BEE_HAPPY` incident with location 120 and radius 1. There will be no other effects on the harvest.

~~When the requirements change after the design review, we will announce a third scenario for which you will need to demonstrate the interaction of your classes using another UML sequence diagram.~~

4. You must draw up a detailed time and work plan for the implementation including testing. This plan should be designed as a group and everyone should be assigned contributions equally. This so-called implementation plan specifies who will be responsible for which parts of the implementation and who will be responsible for testing which components. In this plan, pay particular attention to the dependencies between different components and the order of the next milestones. Do not forget to include writing of the unit, integration, and system tests in the plan as well. A template will be provided in the CMS, and this template is mandatory to use for the submission in your GitLab *design* repository.

Design Review & Design Defense

In the design review, you will receive feedback on your design from a member of the chair. In the design defense, your design has to be approved by a member of the chair. In addition, each group member has to prove their understanding of their group's design. Both appointments are mandatory review appointments on-site at the campus.

After the design reviews, we will introduce changes of the specification that have to be incorporated to your design before the design defense. Try to keep your design modular enough so that any changes can be easily incorporated into your design. However, you should not try to incorporate all possible changes into the design in advance.

[1] We decided on ' as a thousand separator to make larger numbers easier to read.

For both appointments, design review and design defense, note that the current state of your draft that you will present to the chair member must already be uploaded in our GitLab *design* repository 1 hour before your assigned review or defense appointment (see also Submissions). The same holds true for the additional defense, i.e., the design re-evaluation, in case your group did not pass the first one.

1.2. Implementation

In this phase, you will complete the actual implementation of the simulation in your group. You will also write unit tests, integration tests, and system tests and create an implementation report, which have to be pushed to a separate *implementation* repository. Your result must meet the following requirements:

1. Your code must be well structured, well documented and follow the concepts presented in the lecture. We will examine your implementation with the code analysis tool detekt. This happens automatically; your project needs to *build* and pass the code analysis tool on your `main` branch before our tests can be run on your code. Therefore, there can be no problems in this process.

   With the build script we have created for you, you can run this tool yourself from the beginning of the implementation phase to check if your code still has problems.

   In addition, we will manually examine your code to see if you followed higher-level concepts presented in the lecture.

2. Your implementation must correctly implement the specification, for which your implementation must pass the system tests we create. We run our system tests on your implementation regularly during the implementation phase, and the test results will be provided to you at regular intervals. The tests are divided into different categories. For each category, there is a specific subset that your implementation must pass. Additional tests will be provided for further debugging. Once the implementation phase starts, we will release a separate document in which we will specify the mandatory test set.

3. You must implement unit tests, integration tests, and system tests that cover all requirements described in the specification. Your tests are intended to achieve the highest possible code coverage while testing reasonable scenarios, which is ensured by finding mutants with your system tests[2]. To fulfill this criterion, your system tests must pass a specified amount of the released mutants. Additionally, your implementation must pass the unit tests, integration tests, and system tests that you create. Your tests must adhere to best practices, as introduced in the lectures. Your unit and integration tests are there to help you debug and correct errors. In total, your test suite must provide a high coverage of your implementation, especially of the simulation logic.

[2] https://en.wikipedia.org/wiki/Mutation_testing

4. Each individual group member must contribute a significant amount to your group's code for both implementation and test suite.

5. The individual responsibilities and contributions of each group member must be documented in the group's implementation report, which has to be submitted in GitLab *implementation* repository in *markdown* format. This report should attribute the assigned responsibilities as well as the performed contributions of each group member according to the principles taught in the lecture. A template will be provided in the CMS, and this template is mandatory to use for the submission in your GitLab repository.

Code Review

In the code review, a member of the chair will look at your code together with you, point out weak points, and give concrete suggestions for improvement. The code review is also a mandatory review appointment.

Code Submission

For the final submission of your implementation, we will consider the last commit on the `main` branch made to the *implementation* repository no later than 18:00 CEST on the day of code submission. Your submission must include the implementation, the unit and system tests, as well as the implementation report.

Should your group fail, e.g., due to an inadequate implementation, or should we find your individual contribution towards your group project insufficient, you can attend the implementation re-evaluation in the week after the code submission, where you will have to explain and defend your existing contribution to the group project in an individual oral examination in a later specified location on campus.

Attention!

It is not an appropriate contribution to just push a lot of boilerplate code or to create hundreds of very similar or pointless tests.

1.3. Needed Tools

To participate in the SE Lab, you need to bring your own laptop. You need to install the following tools on your device:

- Java 21 (our reference platform uses the latest LTS-version *Temurin® JDK 21.0.8*)[3]
- Version-control system GIT[4]
- IDE (we strongly recommend IntelliJ[5], which is the only IDE in which your tutors will help you with technical issues.)

The build system Gradle, which we use, does not need to be explicitly installed at your end, because we provide you with a project in which a *Gradle wrapper* is already included, which takes care of the installation by itself. Kotlin also does not have to be installed manually but will be installed via Gradle. All other tools you need do not need to be installed, but are already pre-configured in the project by us. All libraries included in the Gradle configuration file (`build.gradle.kts`) may be used. Other libraries must not be used.

Attention!

Be sure here to have GIT properly and consistently configured on all devices you use to commit the code with your full name so that your commits are also assigned to you in our GitLab. In particular, if we cannot identify a person's contribution due to using unclear or multiple user names, we cannot include this contribution in our evaluation.

1.4. Submissions

We will provide you with two *git* repositories for the group phase in our GitLab[6], one for the *design* and one for the *implementation*. To be able to clone or push to the provided repositories, you need an SSH key. You have already learned how to generate an SSH key and which other technical requirements are necessary for using the repositories in the practical tutorial. The entire group will work on those two repositories.

A *design* repository is provided for submitting your design. All design documents and associated diagrams must be pushed to the repository. You can upload photos of your diagrams on paper etc. for this purpose, as long as the resolution is high enough and details are visible.

The preliminary version of your design, which you have to submit prior to the design review, must be pushed and marked with the tag[7] *design_review*, which must be attached to the

[3] https://adoptium.net/de/temurin/releases?version=21&os=any&arch=any
[4] https://git-scm.com/
[5] https://www.jetbrains.com/idea/
[6] https://sopra.se.cs.uni-saarland.de/
[7] https://git-scm.com/book/en/v2/Git-Basics-Tagging

submitted commit revision. This also holds for the final version of your design you must submit for the design defense: Use the tag *design_defense* in this case. In case you fail the design defense, you must submit your revised design under the tag *design_defense_revised*.

To create a tag for the current commit, use the GIT command `git tag <tagname>`. Note that the submission of your design must be pushed to the *design* repository, at latest, 1 hour before your assigned appointment for the design review and defense.

Attention!

All documents that you submit for the design review and design defense have to be provided in well-readable and machine-written *PDF* format. For the implementation plan and report, you must use *markdown* based on the template provided in the CMS.

For your implementation, you must use the `main` branch of the *implementation*. There, you will be provided with a project with a minimal code framework at the start of your implementation phase. We will not consider other branches than `main` for the code submission.

A `results` branch exists in your *implementation* repository. This branch is protected and can not be worked on, as we will provide test results there.

For the final submission of your implementation, the last commit on the `main` branch made to the *implementation* repository no later than 18:00 CEST on the day of code submission counts. It is your responsibility to verify that the push has arrived in the repository and that everything is up to date in a timely manner. Your submissions must be executable on our reference platform (*Debian 12* with *Eclipse Temurin 21.0.8*).

1.5. Use of Tooling

For this project, we do not prohibit you from using any technical tools. However, you are the one responsible for the quality and suitability of their results and how to integrate those into the group project. You must understand your contributions in detail (e.g., code, design, tests) and be able to explain and defend them both to your group members and to the examiners.

To reduce potential conflicts within the group, we encourage each group to discuss and establish their own specific internal rules and processes for using such technical tools.

You must state in your implementation plan for the design phase which technical tools you plan to use and in which aspects those tools shall be involved. You must state in your implementation report for the implementation phase which technical tools you used and in

which aspects those tools were involved (especially any deviations from the statements in the implementation plan).

1.6. Interaction with Tutors regarding Task Specification

Ambiguities in the task specification, which follows in the next chapters, are unavoidable. As in real software projects, there may be situations which are not completely specified in the specification text. Often this holds, for instance, for specific corner cases. In such a case, interaction with the customer is necessary. That is, you need to interact with the tutors (e.g., in the forum) to clarify unspecified details. We will also provide an update thread in the forum[8], where we will post clarifications. Please check this thread and previous forum posts before posting a new question in the forum.

[8] https://forum.se.cs.uni-saarland.de:51443/t/specification-adjustments

2. Saardew Valley: An Organic Farming Simulation

2.1. Introduction

Farming is one of humanity's oldest and most important activities, as it provides the food and raw materials that sustain societies. At its core, it is the process of cultivating plants and managing land to produce a harvest, shaped by the interaction of biological growth cycles, environmental conditions, and labor. Farms can take many forms, from open fields where plants need to be sown annually before harvesting, to plantations where plants are cultivated for many years without human interference. Key farming activities include cutting trees, sowing seeds, controlling weeds, irrigating plants, and harvesting mature crops. These tasks are often supported by machinery to increase efficiency and productivity.

Those environmental influences are a constant factor in farming. Sunlight, soil moisture, and temperature all play crucial roles in plant development, and their balance determines both harvest quality and quantity. Cloud cover can reduce sunlight levels, while rain increases soil moisture. Too much sunlight combined with insufficient moisture can harm plants, just as overly wet conditions can hinder growth. Additional factors such as the availability of pollinating insects, and extreme weather events can also have a major impact.

In recent decades, organic farming has gained significance for its focus on environmental sustainability, soil health, and the avoidance of synthetic pesticides and fertilizers. This approach contributes to biodiversity and long-term soil fertility, but it also brings additional challenges. Pest and weed control must rely on natural methods, and crop growth depends more heavily on favorable environmental conditions, making organic yields more sensitive to changes in sunlight, moisture, and temperature. Additionally, the omission of synthetic fertilizers requires a lot of crop rotation—the practice of growing different crops in succession on the same field. By alternating plant families, farmers can naturally manage soil nutrients, reduce pest build-up, and improve soil structure without synthetic inputs.

This simulation models farming in a simplified way. Time advances in discrete two-week steps, and the simulation can span multiple years. Two types of farmland are included: fields, which ~~can host different plants~~ can host plants of different types over time, and plantations, which grow a fixed plant type. The farm performs essential tasks—sowing, weeding, mowing, cutting, irrigating, and harvesting—using machines. Sunlight and soil moisture are dynamically affected by moving clouds and rainfall, influencing plant growth and finally harvest. By focusing on these selected aspects, the simulation captures key

interactions between farming activities and environmental conditions while omitting more complex, long-term agricultural factors such as soil depletion and fertilization of the soil~~.~~ and crop rotation effects.

2.2. Simulation Resources

In the following sections, we will describe how the map is structured, how the farms and their machines are defined, what kind of plants can be grown, and what kind of incidents can occur during the simulation. Please note that this is only an approximation of the real world and as such might not be coherent with any farms and their strategies in the real world.

2.2.1. Time and Ticks

We will perform a simulation in which multiple actions and incidents happen. For simplicity purposes, we perform the simulation in tick-based manner, where each tick is specified via a tick number, and all actions within this tick are performed consecutively. Each tick lasts a fortnight (two weeks), and a year is simplified as containing 12 months with four weeks each, so that each month consists of two ticks. The time horizon of the simulation is across one or multiple years, as plants require time to grow and be harvested. The simulation can start with any of the 24 ticks within the year period ($\texttt{start\_year\_tick}, \texttt{yearTick} \in 1\text{–}24\,\texttt{tick}$), which is specified at the beginning of the simulation. Table 1 below provides an overview of how the months and ticks correspond to each other.

There will be references to duration or relative ticks in the next sections. Those are to be interpreted as follows: A duration of n ticks means that starting from a specified tick t all ticks up to tick $t+n-1$ are included, but tick $t+n$ is not anymore. In some cases, there is a relative tick mentioned such as after something happened in tick t there must be n ticks before something new can be done. This means that you count starting from tick $t+1$ as the first tick, and only starting from tick $t+n+1$ you can do something new.

2.2.2. Map Configuration

The simulation takes place on a two-dimensional map. The map is used to define the basic setting of the simulation, where the machines of the various farms move, where the plants are located, and where incidents can occur. This map is divided into various tiles that are arranged in a specific layout. Each tile has a unique `id`, a unique 2-dimensional `coordinate`, a `category`, and a set of accessory properties that define the properties of the tile.

Tile Layout

We use a mix of interleaved big octagonal and small square tiles to represent the map. The octagonal tiles are regular. They are positioned directly next to each other, forming a net

Month	Month index	Tick corresponding to the first two weeks (early part) of the month	Tick corresponding to the last two weeks (late part) of the month
January	01	1	2
February	02	3	4
March	03	5	6
April	04	7	8
May	05	9	10
June	06	11	12
July	07	13	14
August	08	15	16
September	09	17	18
October	10	19	20
November	11	21	22
December	12	23	24

Table 1: The assignment of ticks to time periods within the year (`yearTick`)

with square-formed holes. Those are filled with square tiles rotated $45°$, each having the same side length as an octagonal tile. The edges of the octagonal tiles are oriented in the directions $0°$, $45°$, $90°$, $135°$, $180°$, $225°$, $270°$ and $315°$, with $0°$ being the edge on the **north** side of the tile and increasing degrees clockwise. The coordinates start with the origin $(0, 0)$ in an octagonal tile, and continue with even numbers in every direction. For example, the right-adjoining octagonal tile has the coordinate $(2, 0)$ and then the next $(4, 0)$ and so on. The octagonal tile adjoined below the original tile has with the coordinate $(0, 2)$. The square tiles, in contrast, are positioned using odd numbers, with the square below and to the right of the origin starting with $(1, 1)$. Coordinates consisting of both an even and an odd number do not exist. An example of the tiles and coordinates can be found in Figure 1.

Tile Categories

Each tile has a category, and these categories are used to define static, non-changeable properties of the tiles. In this section, we will describe the different categories of tiles that are used in the simulation. Each category of tiles has its own constraints and properties, affecting all aspects of the simulation.

Farmstead `FARMSTEAD` tiles are places in which the farm can store its machines. Only square tiles can be `FARMSTEAD` tiles. Adjoining tiles can be any but `FARMSTEAD` or `MEADOW` tiles. `FARMSTEAD` tiles cover territory owned by farms, and each `FARMSTEAD` tile is owned by exactly one farm.

Field `FIELD` tiles are fields in which farms can seed, grow and harvest plants annually. `FIELD` tiles must lie fallow for 2 complete months (4 ticks) after the tick in which the harvest was reduced to $0\,\mathrm{g}$, until they can be sowed again. `FIELD` tiles are always octagonal

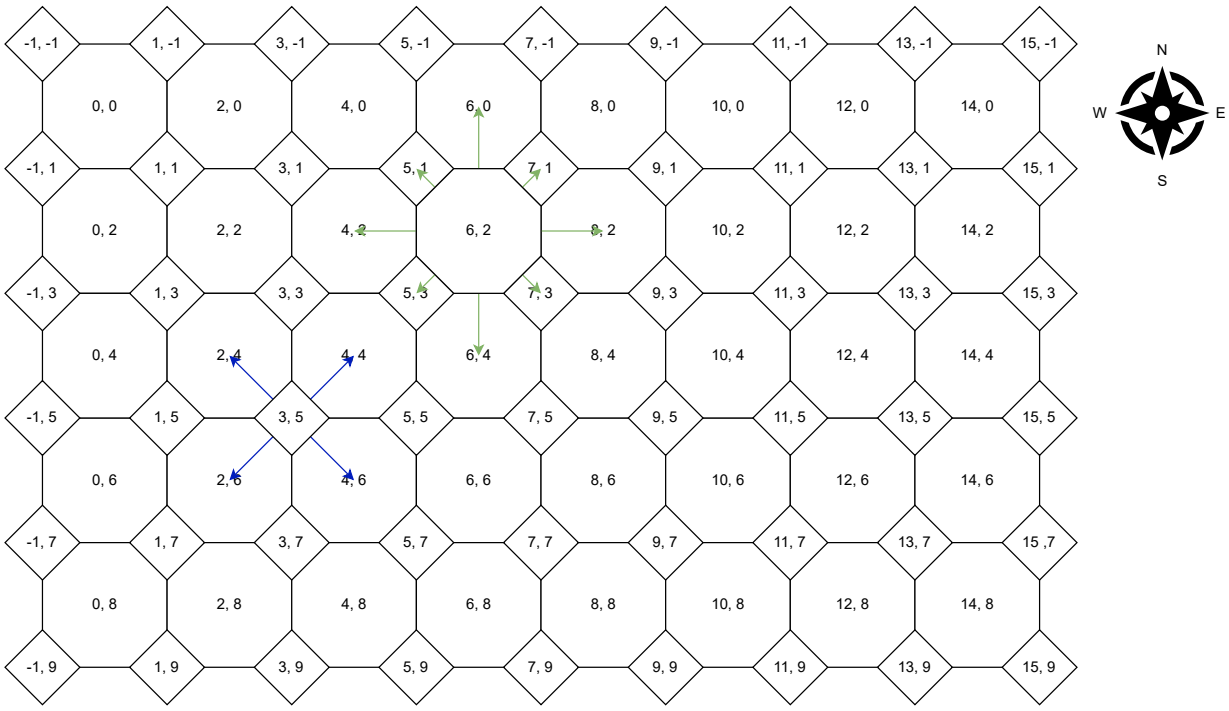


Figure 1: An example of the map layout and coordinates
The blue arrows show the 4 adjoining tiles of a square tile, and the green arrows show the 8 adjoining tiles of an octagonal tile.

and can adjoin all tiles. FIELD tiles cover territory owned by farms, and each FIELD tile is owned by exactly one farm.

Forest FOREST tiles are places where forests grow, and forest animals can originate and cause havoc on adjoining FIELD and PLANTATION tiles. Both square and octagonal tiles can be FOREST tiles. They can adjoin all but VILLAGE tiles.

Meadow MEADOW tiles are places which do not belong to farms and cannot be cultivated. They are commonly between FIELD and PLANTATION tiles where farms let nature reside. Only square tiles can be MEADOW tiles. Adjoining tiles can be any but FARMSTEAD or MEADOW tiles.

Plantation PLANTATION tiles are tiles where farms have grown perennial plants. Only octagonal tiles can be PLANTATION tiles. All tiles can adjoin PLANTATION tiles. PLANTATION tiles cover territory owned by farms, and each PLANTATION tile is owned by exactly one farm.

Road ROAD tiles are tiles that are not used for any action or owned by any farm. They can have any form and adjoin any tile.

Village VILLAGE tiles are places that are inhabited by the population. Both octagonal and square tiles can be VILLAGE tiles. ~~All tiles can adjoin VILLAGE tiles.~~ Adjoining tiles can be FARMSTEAD, FIELD, MEADOW, PLANTATION, ROAD or VILLAGE tiles. They cannot adjoin FOREST tiles.

Tile Accessory Properties

The tiles can have one or multiple accessory properties, some of which are mandatory for a valid map configuration. These properties specify additional information about the tiles that can be used to define the map that is used in the simulation. These properties are static, so they do not change throughout the simulation. In this section, we will describe the different accessory properties that can be assigned to tiles. All properties except for `direction` are mandatory for the tile categories to which they belong, and `direction` is only required if `airflow` exists.

Airflow The `airflow` property belongs to all tiles but VILLAGE tiles, as the smoke in the villages absorbs the clouds, and thus the cloud gets stuck. This property specifies if clouds starting on this field move through the map, influencing the growth of all plants and the state of the harvest. `airflow` always moves the cloud on this tile to an adjoining tile in a specified direction. Thus, any tile with an active airflow also contains a `direction` property.

Direction The `direction` property is part of a tile, and is only present if the `airflow` property is set to TRUE. The `direction` property indicates the adjoining tile to which the clouds on this tile will move. It is represented as an angle, for the square tiles one of $45°$, $135°$, $225°$ and $315°$ and for the octagonal tiles $0°$, $45°$, $90°$, $135°$, $180°$, $225°$, $270°$ and $315°$, and determines the orientation of cloud movements on the tile. The

angle $0°$ represents north, so an airflow in this direction will move any clouds to the north adjoining tile. Similarly, $45°$ moves clouds to the north-east adjoining tile.

Capacity The soil moisture `capacity` property also belongs to tiles that can grow plants (`FIELD` or `PLANTATION` tiles). It specifies the maximum `capacity` of moisture the soil of the tile can hold at any point in time.

Farm The `farm` property belongs to all tiles whose category specifies that they can be owned by farms (i.e., `FIELD`, `PLANTATION`, and `FARMSTEAD` tiles). It specifies the `id` of the farm that owns this tile.

Plant The `plant` property belongs to `PLANTATION` ~~and `FIELD`~~ tiles. It specifies which plant ~~for `FIELD` tiles can grow and for `PLANTATION` tiles~~ is currently growing on this tile. This property is static, thus the plant on each of those tiles will not be changed throughout the simulation. ~~However, `FIELD` tiles still must be sown every season to achieve harvest, but only the plant specified by this property can be sown on this tile.~~ It contains one of the plants `APPLE`, `GRAPE`, `ALMOND`, `CHERRY` ~~for `PLANTATION` tiles and one of `POTATO`, `WHEAT`, `OAT`, `PUMPKIN` for `FIELD` tiles~~.

Possible Plants The `possiblePlants` property belongs to `FIELD` tiles. It specifies which plants can be sowed and grown on this specific tile. In contrast to the `PLANTATION` tile, where the plant does not change, the plants growing on fields can change throughout the simulation. `possiblePlants` are always subset of the following plants: `POTATO`, `WHEAT`, `OAT`, `PUMPKIN`.

Shed The `shed` property belongs to a tile that can host a shed. This property is present in `FARMSTEAD` tiles and specifies whether such a shed is present on this specific tile or not. The sheds can store an arbitrary number of machines and have an unlimited capacity for harvest.

Movement: Traversing Tiles

Clouds can traverse any tile but `VILLAGE` tiles. ~~Machines can traverse all specified tiles.~~ Machines can traverse the `FARMSTEAD`, `FIELD` and `PLANTATION` tiles owned by their corresponding farm. They can traverse any `MEADOW`, or `ROAD` tile, but there are restrictions on traversing `FOREST` and `VILLAGE` tiles. The map may contain „holes“ in the form of unspecified tiles. Those cannot be traversed by machines and clouds.

In case an entity (cloud or machine) attempts to traverse multiple tiles to reach a target tile, there must exist a path made of specified tiles between the start and end tiles on which the entity can traverse from one tile to the next along this path. In case an entity wants to move to a target tile but there is not a valid path to this tile, the entity does not move at all. If an entity has multiple target tiles it can move to, it will only consider the target tiles it can reach. The movement of all entities is instantaneous.

Tile

- id (id ≥ 0) The unique identifier of the tile.
- coordinates ((x, y) with $x, y \in \mathbb{Z}$) The 2-dimensional coordinates of the tile.
- category (category $\in$ {FARMSTEAD, FIELD, FOREST, MEADOW, PLANTATION, ROAD, VILLAGE}) The category of the tile.
- farm (farm ≥ 0) Specifies the farm that owns the tile if the tile is of category FARMSTEAD, FIELD or PLANTATION.
- airflow (boolean) Specifies whether there is an airflow on the tile if the tile is not of category VILLAGE.
- direction (direction $\in \{0°, 45°, 90°, 135°, 180°, 225°, 270°, 315°\}$) the direction of the airflow, in case there is an airflow on the tile.
- capacity (capacity > 0) Specifies the maximum capacity of moisture the soil can hold if the tile is of category FIELD or PLANTATION.
- plant (plant $\in$ {APPLE, GRAPE, ALMOND, CHERRY~~, POTATO, WHEAT, OAT, PUMPKIN~~}) Specifies which plant is currently grown on this tile if the tile is of category ~~FIELD or~~ PLANTATION.
- possiblePlants (array of fieldplant $\in$ {POTATO, WHEAT, OAT, PUMPKIN}) Specifies which plants can be grown on this tile if the tile is of category FIELD.
- shed (boolean) Specifies whether there is a shed on the tile if the tile is of category FARMSTEAD.

Figure 2: Tile properties that are specified in the JSON file

Environment

The environment plays a crucial role in the simulation, influencing the growth of plants on FIELD and PLANTATION tiles. Simulating the environment adds realism and complexity to the simulation, reflecting natural weather phenomena that can significantly impact the distribution and concentration of harvest.

Sunlight

You will have to model the sunlight the plants receive using the average monthly values for sunshine in Saarbrücken[1].

Month	01	02	03	04	05	06	07	08	09	10	11	12
Daily sun hours (monthly average)	7	8	9	10	12	12	12	11	9	8	7	6
Total sun hours per tick	98	112	126	140	168	168	168	154	126	112	98	84

The sunlight is modeled for each tile on which plants can be grown. The sunlight is also influenced by the presence of clouds in a tick. The clouds are dynamic entities that move through the simulation based on the airflow and direction properties defined in the tiles.

During this cloud movement, each tile a cloud traverses (or starts from) loses $3\,\mathrm{h}$ of sunlight for each cloud that traverses the tile during this tick. This does not include the final tile, so the value is not subtracted for when the cloud ends on a tile. Instead, after the cloud movement has been performed, each tile where a cloud is located loses $50\,\mathrm{h}$ of sunlight during this tick. This last loss does not count for clouds that dissipated during this tick. However, the hours of sunlight cannot drop below $0\,\mathrm{h}$, and all further reductions will be ignored.

Clouds

Clouds are entities that move across the map and impact the sunlight and soil moisture of the tiles they traverse. Each cloud has multiple properties.

Id The unique `identifier` of each cloud.

Duration The maximum `duration`, in ticks, before the cloud dissipates. In case the `duration` should be unlimited, it is specified with -1 in the JSON file. Then, the cloud will not dissipate automatically after a given number of ticks, but can dissipate due to other reasons.

Location The `location` on which the cloud will be initialized as given by the `id` of the tile.

Amount The initial `amount` of water within the cloud.

Clouds are created either at the beginning of the simulation as given in the map configuration or by incidents of type `CLOUD_CREATION` specified in the scenario configuration. Clouds will be deconstructed in one of three ways. The clouds will either dissipate after having existed for specified duration of ticks, move onto a `VILLAGE` tile where they get stuck and dissipate immediately, or rain down on a tile until they are empty and dissipate.

There can only exist one cloud on a single tile in a tick. You will have to validate that not more than one cloud will be created at any tile at the beginning of the simulation for the initial cloud configuration. For clouds created by an incident, in case they would be

[1] https://stjerneskinn.com/sonnenaufgang-saarbruecken-neu.htm

located on a tile where a cloud already exists, the newly created cloud and the cloud on the tile merge into one. This merge behavior is identical to the merge behavior during cloud movement.

Cloud

- `id` $(\texttt{id} \geq 0)$ The unique identifier of the cloud.
- `location` $(\texttt{location} \geq 0)$ The `id` of the tile where the cloud is instantiated.
- `duration` $(\texttt{duration} > 0 \;||\; \texttt{duration} = -1)$ The maximum duration, in ticks, before the cloud dissipates.
- `amount` $(\texttt{amount} > 0)$ The initial amount of water within the cloud.

Figure 3: Cloud properties that are specified in the JSON file

Cloud Movement Cloud movement occurs in every tick of the simulation. Each cloud located on a tile with an `airflow` will be move in the `direction` defined in the tile. This means that the cloud moves from the tile it is located on to the adjoining tile indicated by the `direction` (as long as the traversal is possible). The adjoining tile can then move the cloud further, thus clouds can be moved across multiple tiles during one tick. However, each cloud cannot move more than 10 tiles and clouds cannot move onto unspecified tiles.

There can never be more than one cloud on a tile. In case a cloud would be moved onto a tile where a cloud is already located, the two clouds will be merged into one entity with combined features. This new cloud entity will be assigned a new `id` (maximum of all `id`s for clouds that ever existed in the simulation $+1$), the additive `amount` of water of both original clouds, the minimum of both cloud `duration`s starting from this tick as the new `duration`, and the `location` as the `id` of the tile that the cloud would be moved to. For the rest of the current tick, the new cloud can move up to the maximum distance that either of the original clouds were able to move in this tick. The order of movement and merging is specified in the next chapter.

Rain All clouds containing exactly or more than $5'000\,\mathrm{L}$ water can rain down on the tile the cloud is located on. For tiles with soil moisture `capacity`, it only rains if the moisture is below maximum `capacity`. For tiles without soil moisture `capacity`, the rain always occurs and it rains down the whole `amount` of water, which causes the cloud to dissipate. If the difference between current soil moisture and maximum `capacity` is larger than the `amount` of water in the cloud, the cloud rains down its entire water, which again causes the cloud to

dissipate. Otherwise, it rains down an amount just enough to increase the soil moisture to the maximum `capacity` (this amount is removed from the cloud `amount` and added to the soil moisture of the tile), and the cloud continues with its behavior.

Rain occurs only in the cloud phase, and all clouds can rain without moving in this tick, before moving to another tile, or after it has moved onto the current tile. For example, a cloud with $3'000\,\mathrm{L}$ moves to another tile, on which another cloud with $3'000\,\mathrm{L}$ is located. They merge to a new cloud with $6'000\,\mathrm{L}$ on the other tile. As the threshold for rain is reached, the cloud rains down on that tile. The soil moisture on this tile has the maximum of $2'000\,\mathrm{L}$ but is currently at a level of $1'500\,\mathrm{L}$. Therefore, the amount raining down is $500\,\mathrm{L}$ to increase the soil moisture from $1'500\,\mathrm{L}$ to the maximum `capacity` of $2'000\,\mathrm{L}$. Then, the cloud continues moving with $5'500\,\mathrm{L}$ water. In case the tile that the cloud has moved to is not at full soil moisture capacity, it will rain down immediately after the move.

Soil Moisture

The soil moisture of all fields and plantations changes throughout the simulation, starting at maximum soil moisture `capacity`. It slowly reduces over time, in case there are plants growing at the tile at $100\,\mathrm{L}/\,\texttt{tick}$, otherwise at $70\,\mathrm{L}/\,\texttt{tick}$. It is increased by rain (from clouds) or `IRRIGATING` (action performed by farm). The soil moisture impacts the plants growing on this field, as plants require just the right amount of moisture to grow optimally.

Plants

During farming, the plants grow and create harvest that can be collected. Growing plants is a long process involving multiple steps. To keep track with the status of the plants, farms compute a harvest estimate for each planted tile. A farm can calculate an initial harvest estimate for each field or plantation with such plants under optimal conditions. Through the course of the growth process, this number is influenced by multiple factors, some stemming from the environment and others result from the actions of the farm.

The environment-dependent factors are water and sunlight, of which the plants require a certain amount during growth. Regarding the **sunlight**, plants that receive too little starve slowly, while too much sunlight causes them to dry out. For simplification purposes, we only consider too much sunlight harmful. Thus, for each **full** $25\,\mathrm{h}$ of sunlight beyond the plants' comfort, the harvest estimate will be reduced by 10%, which is applied subsequently. For example, $50\,\mathrm{h}$ too much sunlight result in $90\% \times 90\% = 81\%$ of the previous harvest estimate.

The water the plants require is provided by the **soil moisture** (influenced by rain and irrigation). Similar to sunlight, plants require enough but not too much water to thrive. For simplification purposes only too little moisture is considered harmful for plants in this simulation. For each $100\,\mathrm{L}$ below the required amount of water at the tile, the tile loses $50\,\mathrm{g}$ harvest each tick. If the moisture reaches $0\,\mathrm{L}$, the harvest is lost (i.e.,reduced to $0\,\mathrm{g}$).

An additional environmental factor that influences the harvest is **pollination**. To produce harvest for most of the regarded plants, they must be pollinated in a specific time frame. This pollination happens plant-dependent: Some plants can pollinate themselves and are thus independent of outside factors such as insect population, while other plants require insects, and thus, the harvest increases with higher insect population (see subsection 2.2.4).

Next follows a description of the plant life cycle for `FIELD` and `PLANTATION` tiles, especially the actions that the farm must perform to achieve optimal harvest.

Plants on Field tiles In `FIELD` tiles, farms can grow multiple plants: `POTATO`, `WHEAT`, `OAT`, and `PUMPKIN`. As the simulation always starts with fields that do not contain any plants, `SOWING` the plants must first be performed in a plant-specific time frame. The estimated harvest is set to the initial estimate for each plant in the tick where the `SOWING` action is performed. `SOWING` these plants is also possible for up to two ticks after the preferred sowing period, which reduces the expected harvest by 20% per delayed tick. Pollination usually occurs some ticks after `SOWING`, the concrete tick is plant-dependent. After `SOWING`, `WEEDING` should be performed to reduce the presence of weeds and strengthen the plants. If this step is not performed in time, the harvest is reduced by 10% for every tick in which weeding is missed. Finally, the farms are `HARVESTING` from the plants in a specified time frame. The delay of `HARVESTING` severely reduces the harvest that can be collected, however it affects every plant differently.

Potato `SOWING` potatoes should take place in April or May. Initially, the farm calculates a harvest estimate of $1'000'000\,\mathrm{g}$ potatoes per `FIELD` tile. They require high moisture ($500\,\mathrm{L}$) and can withstand a lot of sunlight ($130\,\mathrm{h}$) per tick. They bloom 3 ticks after `SOWING` for 1 tick and can be pollinated by insects, but the harvest is not influenced by pollination. Potato fields require `WEEDING` every second tick after `SOWING`. Additionally, they need `HARVESTING` in September or October. `HARVESTING` at a later tick is impossible, as the frost keeps the potatoes captive in the soil and damages them.

Wheat Wheat commonly has two possible growing periods, one over the winter or another within the year. For this simulation, we focus on growing winter wheat, which requires `SOWING` in October. Wheat requires high moisture ($450\,\mathrm{L}$) but also a bit more shade ($90\,\mathrm{h}$ sunlight) per tick. Initially, the farm calculates a harvest estimate of $1'500'000\,\mathrm{g}$ of wheat per `FIELD` tile. Wheat requires `WEEDING` three and nine ticks after `SOWING`. When wheat blooms in the first tick of May, it self-pollinates, requiring no insects. It requires `HARVESTING` in June or the first tick of July. A later harvest poses the risk of downpour on the fully formed grains, thus the harvest is reduced by 20% for each of the next two ticks, afterward it is a total loss.

Oat `SOWING` oats should take place in the second tick of March. They require medium moisture ($300\,\mathrm{L}$) but also a bit more shade ($90\,\mathrm{h}$ sunlight) per tick. Initially, the farm calculates a harvest estimate of $1'200'000\,\mathrm{g}$ oats per `FIELD` tile. They require `WEEDING` during each of the first three ticks after `SOWING`. Like wheat, oats self-pollinate. `HARVESTING` must take place from July to August. The loss for late `HARVESTING`

is identical to the reduction for WHEAT.

Pumpkin As a plant requiring more warmth and sunlight, pumpkin SOWING happens from the second tick of May until June. Initially, the farm calculates a harvest estimate of $500'000\,\mathrm{g}$ pumpkins per FIELD tile. Pumpkins require high moisture ($600\,\mathrm{L}$) and prefers much sunlight ($120\,\mathrm{h}$) per tick. Pumpkins start blooming 2 ticks after SOWING for 2 ticks and rely on insects for pollination. They require WEEDING every second tick after SOWING and HARVESTING in September and October. HARVESTING at a later tick is impossible, as the frost damages the cells of the pumpkin fruits and thus destroys the harvest.

Plants on Plantation tiles On their PLANTATION tiles, the farms grow trees that bear fruits every year: APPLE, GRAPE, ALMOND, and CHERRY. As the simulation always starts with plants already present in the tiles, the estimated harvest is set to the optimum value at the beginning of the simulation. They require one CUTTING per harvest cycle, usually in fall or spring. If this step is not performed, the number of fruits that can be harvested is halved as the plants invest energy into too many fruits that don't ripen. After CUTTING, the PLANTATION tiles require MOWING to reduce the height of grass and presence of insects. If this step is not performed in time, the harvest is reduced by 10% for each missed MOWING action. Pollination usually occurs in a plant-specific period within each simulated year and can assumed to be successful. HARVESTING the fruits occurs at last in a plant-specific time frame. The delay of HARVESTING severely reduces the harvest that can be collected, however it affects every plant differently.

Apple Apples require low moisture ($100\,\mathrm{L}$) and prefer low sunlight ($50\,\mathrm{h}$) per tick. Initially, the farm calculates a harvest estimate of $1'700'000\,\mathrm{g}$ apples per PLANTATION tile. Apple trees require CUTTING in November or the subsequent February. They bloom in the second tick of April and the first tick of May, where they require insects for pollination. They require MOWING once in early June and once in early September. Apples require HARVESTING in September or early October, HARVESTING a tick later reduces the harvest by half and afterward to $0\,\mathrm{g}$.

Almond Almonds require high moisture ($400\,\mathrm{L}$) and prefer a lot of sunlight ($130\,\mathrm{h}$) per tick. Initially, the farm calculates a harvest estimate of $800'000\,\mathrm{g}$ almonds per PLANTATION tile. Almond trees require CUTTING in November or the subsequent February. Almonds start blooming earlier in the second tick of February until the first tick of March and require insects for pollination. MOWING occurs once in early June and once in early September. Almonds require HARVESTING from the second tick of August until the first tick of October. As the almonds are kept in the center of the stone fruit, HARVESTING a tick later will result in only 10% reduction. Then, the harvest is lost.

Cherry Cherries require low moisture ($150\,\mathrm{L}$) and prefer a lot of sunlight ($120\,\mathrm{h}$) per tick. Initially, the farm calculates a harvest estimate of $1'200'000\,\mathrm{g}$ cherries per PLANTATION tile. Cherry trees require CUTTING in November or the subsequent February. They bloom in the second tick of April and the first tick of May, where they require insects

for pollination. MOWING occurs once in early June and HARVESTING occurs in July. Later harvest is difficult due to the ripeness and fragility of the fruit, thus only 30% remain after a tick and later nothing remains.

Grape Grapes require medium moisture ($250\,\mathrm{L}$) and prefer much sunlight ($150\,\mathrm{h}$) per tick. Initially, the farm calculates a harvest estimate of $1'200'000\,\mathrm{g}$ grapes per PLANTATION tile. Grape plants require MOWING once in early April and once in early July. They also self-pollinate during their blooming in the second tick of June and the first tick of July. They only require a CUTTING in the second tick of July or in August for the grapes to receive enough sunlight. HARVESTING occurs in the first tick of September. For the next three ticks, the harvest loses 5% per tick. This is little compared to other plants, as there are multiple ways of processing the grapes that can also deal with riper fruit.

2.2.3. Farms

In our simulation, we have different farms that each aim to grow plants and produce the best possible harvest. There needs to be at least one farm in the simulation. Farms have various properties. Each farm has a unique `id` and a unique `name`.

Farmsteads The farm owns one or multiple FARMSTEAD tiles on which there may be `sheds` to store machines and harvest.

Fields The farm owns FIELD tiles that the farm can sow and harvest plants in.

Plantations The farm also owns PLANTATION tiles where perennial plants grow, which must only be harvested.

A farm owns at least one FIELD or PLANTATION tile. One farm's FARMSTEAD tiles cannot adjoin any other farm's FIELD or PLANTATION tiles. However, FIELD or PLANTATION tiles owned by one farm can adjoin FIELD or PLANTATION tiles owned by another farm.

Machines The farm owns various `machines`. There are some general-purpose machines as well as some specific for certain kinds of plants. These machines are located in the farm's sheds. A farm owns at least one machine. They can also own machines responsible for plants they do not grow.

Sowing plans The farm sows plants in the FIELD tiles as specified in the `sowingPlans`. Only PLANTS the farm can sow can be used in `sowingPlans`.

There can be multiple farms in the simulation with an overlapping in the plants they sow or grow. Each farm has global knowledge about their fields, plantations, and machines.

Machines

There exist multiple machines that farms use to operate on the fields. Each machine has a unique `id` and `name`. All machines have specific `actions` they can perform according to

Farm

- `id` (`id` ≥ 0) The unique identifier of the farm.
- `name` (string) The unique name of the farm.
- `farmsteads` (array of `tileID` ≥ 0) The `id` of the `FARMSTEAD` tiles the farm owns.
- `fields` (array of `tileID` ≥ 0) The `id` of the `FIELD` tiles the farm owns.
- `plantations` (array of `tileID` ≥ 0) The `id` of the `PLANTATION` tiles the farm owns.
- `machines` (array of `machine`) The machines the farm owns.
- *sowingPlans* (array of `sowing plan`) The sowing plans the farm intends to fulfill.

Figure 4: Farm properties that are specified in the JSON file

the life-cycle requirements of field and plantation plants. In addition to `SOWING`, `WEEDING`, `HARVESTING`, `CUTTING` and `MOWING` as described in 2.2.2, the machines can also spend their time `IRRIGATING` on `FIELD` or `PLANTATION` tiles to regulate the soil moisture. Each machine also has a specific set of `plants` they can be used on. All machines have unlimited supply and capacity for seeds, harvest and water, they don't need to be refueled or emptied. Machines cannot traverse `FOREST` tiles as the trees stand too close together. Additionally, machines containing harvest cannot traverse `VILLAGE` tiles as the precious cargo would drop on the streets. This can lead to machines not being able to return to the shed they originate from.The machines of one farm cannot cross another farm's `FARMSTEAD`, `FIELD` or `PLANTATION` tiles.

Actions Determines the `actions` the machine can perform, which is a subset of `SOWING`, `CUTTING`, `MOWING`, `WEEDING`, `HARVESTING`, `IRRIGATING`. There must always be at least one action.

Plants Determines the `plants` this machine can be used on, which is a subset of the field and plantation plants. There must always be at least one plant.

Duration Determines the `duration` the machine takes to perform its action on one tile. This is measured in days.

Location The `location` of the shed the machine is initially stored at given in form of the

tile id.

Machine

- id (id ≥ 0) The unique identifier of the machine.
- name (string) The unique name of the machine.
- actions (array of action $\in \{$SOWING, CUTTING, MOWING, WEEDING, IRRIGATING, HARVESTING$\}$) The actions the machine can perform.
- plants (array of plant $\in \{$POTATO, WHEAT, OAT, PUMPKIN, APPLE, GRAPE, ALMOND, CHERRY$\}$) The plants the machine can operate on.
- duration ($0 <$ duration ≤ 14) The number of days a machine requires to perform one action on one tile.
- location (tileID ≥ 0) The initial location of the machine.

Figure 5: Machine properties that are specified in the JSON file

Sowing Plans

The SOWING of plants on FIELD tiles does not occur automatically whenever possible. Instead, farms plan their sowing in accordance with an association for organic farming, so that the farms vary in the plants they sow and the farms have a better chance of selling their harvest on the food market for a good price. They create sowing plans for the farms, which specify when and where farms can start SOWING which plants.

Each sowing plan has an unique id, and a tick in which (starting from tick 0) the farm should start SOWING. Another property plant specifies which plant is to be sowed. The location of sowing is given either with the property fields, in which an array of ids referencing to FIELD tiles is stored, or using the properties location and radius, which specify a central tile and the number of tiles around this tile to sow (the measure of radius is defined as in subsection 2.2.4). In the latter case, location does not have to be a FIELD tile, but at least one tile in the area specified by location and radius must be a FIELD tile.

SOWING according to the sowing plan has the highest priority of all actions the farm can take. A sowing plan can only be active (acted upon) if the simulation tick is equal or greater to the plan's tick. It is however possible that a sowing plan cannot be fulfilled, because it is not the time to sow this plant or no machine is available. It might be that not all FIELD tiles

included in the sowing plan are ready for SOWING, if a tile is already planted or hasn't been fallow for long enough. Then the farm are SOWING only on the ready tiles. A sowing plan counts as fulfilled if at least one tile was sowed. Then, it is removed. Otherwise, the plan will be tried again every tick. It must be validated at the beginning of the simulation that the tiles affected by the sowing plan include at least one tile that is a FIELD tile throughout the entire simulation. This means the type of the tile is not changed by an incident at any tick, even after the simulation time exceeds the maximum number of ticks.

Sowing plan

- id (id ≥ 0) The unique identifier of the sowing plan.
- tick (tick ≥ 0) The tick in which the sowing plan is activated.
- plant (fieldplant $\in \{$POTATO, WHEAT, OAT, PUMPKIN$\}$) The plant the farm must sow.
- fields (array of tileID ≥ 0) The id of the FIELD tiles the plant should be sowed at.
  This is used in case there is no location and radius.
- location (tileID ≥ 0) The id central tile around which to sow the plant at.
- radius (radius ≥ 0) The radius around the central tile that indicated how many tiles shall be sowed at, measured in tiles.

Figure 6: Sowing plan properties that are specified in the JSON file

Actions

The farms operate by performing any amount of a set of possible actions. All actions are performed by machines. Thus, for the farm to do something, you must identify an action, a machine, and a tile as specified in the paragraphs below, so that this machine can perform this action on this tile. This machine will then exit its shed and perform this action on the specified tile. Then, the machine will continue performing any suitable actions on other tiles and try to return to a shed. Only after this machine has tried to return to a shed, the next action, the next tile, and the next machine will be selected according to the prioritization, and this machine will exit the shed and perform its actions.

Prioritization The priority of all possible actions that a farm can perform on any tile and with any machine is as follows: ~~The first priority in any farm is to sow plants in all possible FIELD tiles, in the order of POTATO, WHEAT, OAT, PUMPKIN.~~ The first priority in any farm is to fulfill any active sowing plans. Among active sowing plans for a farm, the plans are chosen in order of increasing `tick` and then `id`. For each plan, all tiles to sow are extracted and the machines are assigned one after the other, until either all tiles ready for SOWING have been sowed with the `plant` specified in the plan, or there are no machines left that can perform the SOWING of this `plant` on the remaining tiles. Next is the HARVESTING of any ripe plants on the PLANTATION tiles. Then, they start HARVESTING any ripe plants on the FIELD tiles. Next, the machines start CUTTING any PLANTATION tiles. Finally, the remaining actions are decided on machine level: From lowest to highest `id` of the machine, the machines first start IRRIGATING then WEEDING all FIELD tiles. Next, IRRIGATING and then MOWING are performed on all PLANTATION tiles.

Prerequisites For each action, it must first be checked whether the tile(s) require an action (and the action can be performed on the tile), whether the right machines are available for this action and the given plant on the tile and if the machine can reach the tile. Only then does the machine "move" to the tile and perform the given action. There are some action-specific constraints: For example, each FIELD tile requires a fallow period of 4 complete ticks between HARVESTING and SOWING new plants. Additionally, SOWING occurs only if triggered by a sowing plan. If the harvest has been reduced to $0\,\mathrm{g}$ in this FIELD tile, the plants and fruits do not have to be removed before SOWING new plants. Another constraint is given for IRRIGATING. IRRIGATING is only required on tiles where the moisture is less than needed by the plant growing on the tile. It is also required if the harvest has been reduced to $0\,\mathrm{g}$ or a FIELD tile is fallow, unless the tile is a PLANTATION tile permanently disabled by the drought incident. Once on the tile, they fill it with moisture until the maximum `capacity` for soil moisture is reached. Other actions have less requirements: All actions except for IRRIGATING require the right time period for the plants growing in a tile. Except for IRRIGATING and SOWING, all actions require growing plants on their tile, which means that the harvest estimate must be greater than $0\,\mathrm{g}$.

Tile and Machine Selection The tiles for each of those actions are prioritized in ascending `id` of the tiles, thus the tiles with lowest `id` are selected first. In case multiple machines are available for an action that is not yet bound to a specific machine on a specific tile, it must first be checked which machines can perform the action and which machines can reach the tile. Among those machines, the machine with the lowest `duration` and then the lowest `id` is selected.

Action Continuation If a machine has completed its action on a tile before the tick ends, the machine can continue performing the **same** action on other tiles that contain FIELD or PLANTATION tiles owned by this farm, as long as this new tile is reachable within traversing two tiles from the tile on which the machine has completed its action. However, SOWING

can only be continued for the same sowing plan and `HARVESTING` can only be continued for the same plant. If there are multiple available tiles, the tile with the lowest `id` is selected. However, if such an action on a new tile would take more time than is remaining in the tick, the machine will not try to move there and perform this action. Thus, the machine may move up to 26 tiles between actions within a tick (if the duration is 1 and there are 14 tiles with distance 2 on which the machine can perform the same action).

Post-Action Each machine performs only one type of action (on one or multiple tiles). After the actions have finished, a machine always tries to return to a shed, where it unloads the collected plants. This shed is, if possible, the shed of the farm that the machine last exited. In case the machine cannot return (because it cannot traverse `VILLAGE` tiles when containing harvest), another reachable shed of the farm (with lowest `id` for the tile) is the return destination. In case a machine cannot return to any of the farm's sheds, it remains on the current tile and the machine will not be considered again for the rest of the simulation. There are no parallel actions on a tile, if a tile has been operated on once in this tick (e.g., to harvest), no other action can be performed in this tick. If a machine has already been used in a tick and returned to the shed, it will not perform any other action in this tick.

Hint for implementation

It is clear that the described operations do not perform optimally. This is accepted to deliver a both deterministic and rather simple, greedy algorithm (compared to solving a planning problem and optimizing the harvest return).

2.2.4. Incidents

In our simulation, we have different incidents that can occur during the simulation. They can have different impacts on the simulation and can influence the map, farms and harvest. Each incident has a unique `id`, an incident `type`, a `tick` when the incident occurs, and a set of properties that define the impact of the incident on the simulation. Some incidents have a `location` property that states the central tile for an incident, and a `radius` property that defines the surrounding tiles on which the incidents shall be applied. This `radius` property is defined recursively: It is set to 0 if the incident only affects the tile mentioned in the `location` property. An increase of 1 in the `radius` property means that all tiles adjoining the ones already affected are also affected by the incident. If an incident has those properties, it is explicitly stated in the description below. The incidents are known globally upon their occurrence.

Cloud Creation

A cloud creation is an incident that impacts a specified set of tiles. It lasts only for a single tick, in which it creates cloud instances. Therefore, the incident has a `location` property that states the central tile where the main cloud must be instantiated, and a `radius` property that defines the surrounding tiles where clouds will also be created. Clouds cannot be created on `VILLAGE` tiles. Thus, it must be validated during the initialization of the simulation that at least one of the tiles is not a `VILLAGE` at the moment of incident. Furthermore, there cannot be multiple clouds created on one tile at the same tick.

Additionally, cloud creation has an `duration` property containing a positive number, which states for how long the clouds exist at maximum. In case the cloud shall exist for an unlimited number of ticks, the `duration` is specified with -1 in the JSON file. Finally, cloud creation has an `amount` property that defines the amount of water the clouds contain at the tick of creation. As an `id`, the newly instantiated clouds receive the highest cloud `id` that previously existed plus one (in case there is none, start with 0). In case of multiple clouds created, the new clouds are created and `id`s are given in order of ascending tile `id`. If a cloud would be created on a tile where a cloud already exists, the two merge immediately upon creation before another cloud will be created in the next tile. Upon instantiation, the new clouds do not affect the environment during this tick (no movement, sunlight or rain). The duration coun ts only from the next tick, so a duration of 1 means that the cloud will dissipate in the next tick.

Animal Attack

Many animals living in the surrounding `FOREST` tiles struggle with finding enough food and water to survive. As their habitat shrinks, they become desperate to find food and encroach on the vast `FIELD` and `PLANTATION` tiles. An animal attack is an incident that impacts a tile and a specified radius around the tile, for which it has a `location` and `radius` property. It must be validated during the initialization of the simulation that at least one of the impacted tiles is a `FOREST` at the moment of incident. It lasts only for a single tick, in which it causes animals to break out from all impacted `FOREST` tiles and attack adjoining `FIELD` and `PLANTATION` tiles. They stampede through `FIELD` tiles and eat the plants, which causes massive damage for each incident. Therefore, the harvest in `FIELD` tiles is reduced to half. As `GRAPE`s are hanging low on the plants, the effects are as devastating, so the harvest is reduced to half. Other plantation trees such as apples, almonds and cherries are less affected. Here, the animals eat the grass, which resets the need for `MOWING` in this tick and the next, and eat 10% of the fruits. The effects occur only once per incident and per `FIELD` and `PLANTATION` tile, no matter how many affected `FOREST` tiles are adjoining.

Bee Happy

Many farms are conscious of their impact on the environment and decide to leave part of their acquired territory as meadows between their planted tiles, in which animals such as wild bees or rabbits can find their home. This also has a positive effect on the harvest in surrounding areas, as the wild bees pollinate the plants and thus increase the harvest. Bee happy is an incident that impacts a tile and a specified radius around the tile (for which it has a `location` and `radius` property). It lasts for a single tick in which the bees from all `MEADOW` tiles within the affected tiles swarm the `FIELD` and `PLANTATION` tiles within 2 tile distance from those `MEADOW` tiles and excessively pollinate the plants. In case the plants on such a tile require pollination by insects and the plants are blooming, this leads to an increase in harvest specified in percent by the `effect` property. There must be at least one `MEADOW` tile in the affected tiles. The effects occur only once per incident and per `FIELD` and `PLANTATION` tile, no matter how many affected `MEADOW` tiles are within 2 tile distance. There can be multiple incidents of this type, both within a tick and within the blooming phase of a plant.

Drought

Facilitated by the climate change, extreme weather phenomena occur increasingly. They pose a gigantic danger to farming and thus feeding the population, as they have a negative effect on any plant life. Droughts are incidents that impact a tile and a specified radius around the tile (for which it has a `location` and `radius` property). It lasts for a single tick in which the soil moisture is reduced to $0\,\mathrm{L}$ and the plants on an affected `FIELD` or `PLANTATION` tile die. In the subsequent computation of the harvest estimate, this reduces the harvest to $0\,\mathrm{g}$ and permanently kills all plants on every affected `PLANTATION` tile. After this event, the affected `PLANTATION` tiles will not be considered for any actions and will yield no harvest, but the farm's machines and all clouds can traverse the tiles without problems. In contrast, the affected `FIELD` tiles can be considered for `SOWING` after lying fallow for the required time period. There must be at least one `FIELD` or `PLANTATION` tile in the affected tiles.

Broken Machine

To work on farms this large and generate gigantic amount of food, the farms rely on machines to perform the major workload. However, those machines can break due to age, wear and tear, or excessive labor. This means that a machine cannot perform any more work and requires repair. The broken machine incident impacts a machine specified by the `machine id` property for a number of ticks (>0) given by the `duration` property. This duration counts from the start of the next tick, so a duration of 1 means that the machine can't be used in the next tick, but again in the tick after. In case the machine cannot be repaired, the `duration` is set to -1.

City Expansion

Some villages in the simulation thrive so much that they need to expand their territory. `ROAD` or `FIELD` tiles adjoining villages may be transformed into `VILLAGE` tiles, cutting off place to grow farms or places for traversing machines. For this, the incident has a property `location` that describes the tile that will be transformed into a `VILLAGE` tile. In case a stuck machine is located on this tile, the machine can simply stay there. If this is a `FIELD` tile with plants, the plants will be destroyed and cannot be harvested. All properties not required for the new `VILLAGE` tile will be further ignored, and the tile will from now on behave exactly as any other `VILLAGE` tile. You must validate at the initialization of the simulation that, at the point of this incident, the new `VILLAGE` tile will adjoin at least one existing `VILLAGE` tile.

Incident

- `id` ($\texttt{id} \geq 0$) The unique identifier of the incident.
- `type` ($\texttt{type} \in \{\texttt{CLOUD\_CREATION}, \texttt{ANIMAL\_ATTACK}, \texttt{BEE\_HAPPY}, \texttt{DROUGHT}, \texttt{BROKEN\_MACHINE}, \texttt{CITY\_EXPANSION}\}$) The type of the incident.
- `tick` ($\texttt{tick} \geq 0$) The tick when the incident occurs.
- `duration` ($\texttt{duration} > 0 \;||\; \texttt{duration} = -1$) The duration of the incident in case it is a cloud creation or broken machine.
- `location` ($\texttt{location} \geq 0$) The `id` of the tile where the incident occurs in case it is a cloud creation, animal attack, bee happy, drought or city expansion.
- `radius` ($\texttt{radius} \geq 0$) The radius around the affected tile in case it is a cloud creation, animal attack, bee happy or drought, measured in tiles.
- `amount` ($\texttt{amount} > 0$) The total amount of water the cloud shall contain in case it is a cloud creation.
- `effect` ($\texttt{effect} > 0$) The effect on harvest of increased pollination in case it is a bee happy.
- `machineid` ($\texttt{machineID} \geq 0$) The `id` of the machine that is removed from the map in case it is a broken machine.

Figure 7: Incident properties that are specified in the JSON file

2.2.5. Computing the Harvest Estimate

As the harvest depends on many factors over ticks, the harvest estimate must be computed every tick for every tile. The start is always the base value at the beginning of the tick. Based on this value, the computation increases or decreases the estimate using a fixed set of rules.

The computation always relies on the current estimate. If we start with a current estimate of $5'000\,\mathrm{g}$ and apply the effect of $55\,\mathrm{h}$ too much sunlight in a tick, this first causes a reduction of 10% (for $25\,\mathrm{h}$) on the current estimate. This results in a new current estimate of $4'500\,\mathrm{g}$. A subsequent reduction of 10% (for further $25\,\mathrm{h}$) will cause a new estimate of $4'050\,\mathrm{g}$. Next, the effect of having too little soil moisture might have to be applied, which would be calculated based on the current estimate of $4'050\,\mathrm{g}$.

Some rules only apply if an action should have been and was performed in this tick (e.g., `SOWING`), and the other rules apply if an action should have been performed but it was not (e.g., `WEEDING`, `CUTTING`, `HARVESTING`). An action counts as missing or late if it was not performed at any tick within the preferred time period for the plant and the penalty is applied at the end of the last tick within this time period for each harvest cycle. For actions that have no late period, the action is immediately seen as missing. For actions that can be performed late, the penalty for lateness is applied every tick, until the end of the late period, where the effect of the action missing is applied. The rules are applied in the following order:

For `FIELD` tiles, the estimate is set to the initial value immediately after `SOWING` (in case `SOWING` was performed). Then, the effect of `SOWING` **late** (in case the `SOWING` was performed after the preferred time period) is applied, as this affects the initial estimation. Afterward, the influence of the environmental factors, first **sunlight** and then soil **moisture** are considered. If the tile received too much sunlight for the plant, or had too little moisture, the estimate is adapted. Next, the effect of not `WEEDING` a tile when it is required by the plant is added to the calculation. Then, the effect of not `HARVESTING` after the harvest period has ended is taken into account.

Regarding `PLANTATION` tiles, the estimate is set to the initial value both at the beginning of the simulation and at the beginning of every November ($\texttt{start\_year\_tick} = 21$). Next, the environmental factors, first **sunlight** and then soil **moisture** are included. Then, the effect of missing or late actions is calculated in the order `CUTTING`, `MOWING`, `HARVESTING`.

After all actions and environmental factors have been considered, the influence of incidents is calculated. These incidents impact the harvest estimate in order of execution.

Any such adjustments are calculated on integer numbers and follow set rules (e.g., losing 10%). In case the resulting number is no longer an integer, the result is rounded downward to the next integer number. Additionally, all harvest is non-negative: Reductions beyond $0\,\mathrm{g}$ set the harvest to $0\,\mathrm{g}$.

After `HARVESTING` a tile, the harvest estimate is set to $0\,\mathrm{g}$. Within a tick, `HARVESTING` occurs before the computation of the harvest estimate, thus a machine is always `HARVESTING` the

amount given by harvest estimate of the previous tick.

At the beginning of the simulation, you have to make sure that the harvest estimate for plantation plants reflects all penalties for late harvest that would have been applied in previous ticks.

2.3. Simulation Details

In this section, we will describe the overall behavior of the simulation, and the behavior of the individual farms during the simulation. We also describe what information you need to log so that we can assess your implementation from the outside.

2.3.1. Logging

As there is a lot of information that the simulation needs to store and keep track of, we use log levels to allow specific logging of information. Commonly used to specify different levels of severity, we use the log levels here to log information at different levels of detail. This also helps you during testing: If you want to create a sytem test that spans multiple ticks, you can test with high-level information and don't have the huge log overhead. In contrast, you might want to validate the functionality for specific situations or corner cases, for which you can use the detailed log information to catch bugs and errors early on. We will use the following three log levels:

- `DEBUG`: With this log level, all of the logs specified in this chapter will be output.
- `INFO`: With this log level, we remove all logs that do not report activities of the simulation.
- `IMPORTANT`: With this log level, we only include the most pertinent activities of the simulation.

This log level will be specified in the command line via the argument `log_level`.

In the following log descriptions, every log statement will have a log level assigned. It is crucial to remember that `INFO` logs will also be output if a log level of `DEBUG` is specified, and that `IMPORTANT` logs will be output no matter the log level.

Log statements usually contain variable part, indicated by $, where you have to fill in the required information. Often, this is just one piece of information you can just state, but occasionally, there might be multiple pieces of information you must describe in one variable part (e.g., multiple tile `ids`). In this case, the information must be sorted in ascending order and output separated by commas (e.g., $3,5,7,8$).

2.3.2. Initialization

Before the simulation starts, the different configuration files have to be parsed and validated to initialize the simulation. There are three different configuration files:

- `map` defines the tiles for the map, and their different properties.
- `farms` defines the farms, their machines, and their different properties.
- `scenario` defines the simulation parameters, that is all clouds at the beginning of the scenario and all incidents that can occur during the simulation.

Additionally, the user has to provide both the start tick within the year period and the maximum number of ticks the simulation should run for as a command line argument (c.f.,Section 3.2). The schemes for the JSON files can be found inside the repositories in the folder `src/main/resources/schema`. Please keep in mind that the simulation (and the information in the scenario file) still starts with tick 0 and increases with each tick until the maximum tick is reached. The start tick within the year is given as ($\texttt{start\_year\_tick} \in 1\text{–}24\,\texttt{tick}$) and only serves the purpose of communicating when within the year the simulation will start. For example, if the simulation starts in the first tick of January, the simulation tick starts with 0 and `start_year_tick` starts with 1, but if the simulation starts in the first part October, the `start_year_tick` is 19, but the simulation still begins with tick 0.

Configuration Parsing The three configuration files will be provided as JSON files. You can find more details on the structure of the JSON files in Section 2.2. After parsing the JSON files, your program has to validate the configuration files according to the rules defined in Section 2.2. Each configuration file has to be validated separately, and the simulation should not start if any of the configuration files is invalid. Some constraints can be checked isolated for the different configuration files, but some constraints can only be checked in the context of the other configuration files.

The order of parsing and validation is as follows:

1. Parse and validate the `map` configuration.
2. Parse and validate the `farms` configuration.
3. Parse and validate the `scenario` configuration.

After successfully parsing and validating each configuration file, your program has to output to the output handle (which is provided as command line argument when starting the simulation) a log message containing the type of the log message (i.e., `Initialization Info`) and the name of the configuration file (where `$filename` is a placeholder variable for the name of the configuration file).

```
[INFO] Initialization Info: $filename successfully parsed and validated.
```

In case the configuration file is invalid, your program has to output to the output handle a log message containing the type of the log message (i.e., `Initialization Info`) and the name of the configuration file.

```
[IMPORTANT] Initialization Info: $filename is invalid.
```

Simulation Initialization After successfully parsing and validating the configuration files, the simulation can be initialized. To initialize the simulation, you have to create the map, the farms, and the scenario. The map is created based on the parsed and validated `map` configuration. To create the farms, you have to use the information from the parsed and

validated `farms` configuration and create the farms with their machines. The scenario starts always with tick 0 and the cloud and further incidents are based on the parsed and validated `scenario` configuration.

Simulation Start After the initialization of the simulation, the simulation can start. The simulation is divided into ticks, and each tick represents one time unit. In each tick, the farms can move and act using their machines. Additionally, the clouds move according to the air current of the tiles and incidents can occur during the simulation. To simplify the simulation, we assume that the simulation is deterministic, and the order of the farms is fixed. That means, while technically the actions from farms, the clouds, and the incidents happen simultaneously during a tick, we assume that the farms, clouds and incidents act in a fixed order. In addition, we assume that during a tick, the rules for all actions do not operate on the state at the end of last tick, but at the current tick including the state from everything that happened before in the current tick.

To start the simulation, your program has to output to the output handle a log message containing the type of the log message (i.e., `Simulation Info`), the tick within a year that is regarded as stated in Table 1 and the message that the simulation started.

```
[INFO] Simulation Info: Simulation started at tick $yearTick within the year.
```

2.3.3. Simulation Behavior

The general behavior of each tick during simulation is as follows:

1. Start of the tick.
2. The reduction of the soil moisture.
3. The clouds move one after another in ascending `id`.
4. The farms act one after another in ascending `id`: They assign actions to machines and then let the machines perform their action, influencing the growth of the plants and the resulting harvest.
5. Incidents which start or end at this tick occur during the simulation.
6. Compute harvest estimate of plants.
7. End of the tick.

Before the beginning of each tick, you have to check whether the simulation has reached the maximum number of ticks. In case the simulation has reached the maximum number of ticks, the simulation has to output to the output handle a log message containing the type of the log message (i.e., `Simulation Info`) with the current tick and the message that the simulation ended.

```
[IMPORTANT] Simulation Info: Simulation ended at tick $tick.
```

Otherwise, at the beginning of each tick of the simulation, you have to output to the output handle a log message containing the type of the log message (i.e., `Simulation Info`) with the current tick and the tick within the year (c.f. Table 1) and the message that the tick started.

```
[INFO] Simulation Info: Tick $tick started at tick $yearTick within the year.
```

After the tick has started, the environment is simulated.

Moisture Reduction At the beginning of the tick, the moisture is reduced in all `FIELD` and `PLANTATION` tiles. As only cases with less moisture than required by the plants on planted tiles have a direct impact on the simulation, the simulation has to output to the output handle a log message containing the type of the log message (i.e., `Soil Moisture`) with the amount of `FIELD` and `PLANTATION` tiles that are planted and where the moisture is below the threshold the plants require, and the message that the soil moisture is below the threshold.

```
[INFO] Soil Moisture: The soil moisture is below threshold in $amountField FIELD and $amountPlantation PLANTATION tiles.
```

Clouds Next is the cloud phase, where clouds will rain, move, unite and dissipate. The logs are printed in this order for each cloud and each cloud movement between two tiles, only then does the next cloud (movement) start.

In case a cloud contains enough water to rain, the cloud rains down on the tile below. This can occur before or after each move or union. For this, the cloud has to output to the output handle a log message containing the type of the log message (i.e., `Rain`) with the `id` of the cloud, the `id` of the tile where the cloud rains, the amount of water that rains down and the message that the cloud rains.

```
[IMPORTANT] Cloud Rain: Cloud $cloudID on tile $tileID rained down $amount L water.
```

Each cloud moves according to the `airflow` of the tile it is located on, to the tile the airflow `direction` points to if the cloud can move there. The order of the cloud movement is from the lowest to the highest `id` of the cloud. After the cloud has moved, the cloud has to output to the output handle a log message containing the type of the log message (i.e., `Cloud Movement`) with the `id` and the `amount` of water of the cloud, the `id` of the tile the cloud moved from, the `id` of the tile the cloud moved to, and the message that the cloud moved.

```
[INFO] Cloud Movement: Cloud $cloudID with $amountFluid L water moved from tile $startTileID to tile $endTileID.
```

Additionally for clouds originating from FIELD and PLANTATION tiles, the message prints the current amount of sunlight for the tile the cloud moved from in this tick.

```
[DEBUG] Cloud Movement: On tile $startTileID, the amount of sunlight is $amountSunlight.
```

In case a cloud moved onto a tile where another cloud existed and they united as one new cloud on this tile, the clouds have to output to the output handle a log message containing the type of the log message (i.e., Cloud Union) with the ids of the original clouds, the id, duration, and amount of water of the new cloud, the id of the tile where the union occurred, the and the message that the clouds united.

```
[IMPORTANT] Cloud Union: Clouds $cloudIDFromTile and $cloudIDMovingToTile united to cloud $cloudIDNew with $amount L water and duration $duration on tile $tileID.
```

In case the cloud moved onto a VILLAGE tile and got stuck, the cloud has to output to the output handle a log message containing the type of the log message (i.e., Cloud Movement) with the id of the cloud, the id of the VILLAGE tile where the cloud got stuck, and the message that the cloud got stuck.

```
[INFO] Cloud Dissipation: Cloud $cloudID got stuck on tile $tileID.
```

At the end of the movement for each cloud, there exist two further reasons for this cloud to dissipate: rain and maximum duration. In case either holds true, this cloud has to output handle a log message containing the type of the log message (i.e., Cloud Dissipation) with the id of the cloud, the id of the tile where the cloud is located and the message that the cloud dissipates. For clouds that have reached maximum duration, the message is logged immediately after this cloud's movement. For clouds dissipating due to rain, the message is printed immediately after the log about this cloud's rain.

```
[INFO] Cloud Dissipation: Cloud $cloudID dissipates on tile $tileID.
```

After all clouds have performed their actions, the FIELD and PLANTATION tiles with a cloud located on them have to output to the output handle a log message in increasing order of tile id with the id of the tile, the id of the cloud, the amount of sunlight of the tile and the message that a cloud located on the tile affects the sunlight.

```
[DEBUG] Cloud Position: Cloud $cloudID is on tile $tileID, where the amount of sunlight is $amountSunlight.
```

Farms After the clouds have moved, the farms can move and perform actions. The farms act in order from the lowest id to the highest id, and each farm finishes all its actions before the next farm starts.

The farms now start moving their machines and performing actions. Before the farm can act, it has to output to the output handle a log message containing the type of the log message

(i.e., Farm) with the id of the farm and the message that the farm is to act.

```
[IMPORTANT] Farm: Farm $farmID starts its actions.
```

Additionally, the farms state their intended SOWING activity by outputting to the log handle a log message containing the type of the log message (i.e., Farm) with the id of the farm, the ids of the active sowing plans and the message that the farm plans to sow according to those plans if possible. This message is also sent if there are no active sowing plans.

```
[DEBUG] Farm: Farm $farmID has the following active sowing plans it
intends to pursue in this tick: $sowingPlanIDs.
```

Each farm can perform one or multiple of each of those actions, depending on the number of machines they have at their disposal, in the same tick: SOWING, CUTTING, MOWING, WEEDING, HARVESTING, IRRIGATING. The actions per farm occur in the order as described in subsubsection 2.2.3. As the actions are assigned per machine, the log must first convey that the machine performs an action. The common log message that must be output to the output handle a log message contains the type of the log message (i.e., Farm Action) with the type of action, the id of the machine, the id of tile the machine moves to, and the duration this action takes.

```
[IMPORTANT] Farm Action: Machine $machineID performs $actionType on
tile $tileID for $duration days.
```

In case the action was SOWING, it has to output to the output handle an additional log message containing the type of the log message (i.e., Farm Sowing) with the id of the machine and the plant sowed, according to which sowing plan, and the message that the machine sowed.

```
[IMPORTANT] Farm Sowing: Machine $machineID has sowed $plant according
to sowing plan $sowingPlanID.
```

In case the action was HARVESTING, it has to output to the output handle an additional log message containing the type of the log message (i.e., Farm Harvest) with the id of the machine, the plant and the amount of harvest collected, and the message that the machine has collected harvest.

```
[IMPORTANT] Farm Harvest: Machine $machineID has collected $amount g of
$plant harvest.
```

These log outputs starting from the type Farm Action are repeated for every action a machine performs.

After the machine has performed all actions, it tries to return to the shed. In case the return was successful, it has to output to the output handle a log message containing the type of the log message (i.e., Farm Machine) with the id of the machine, the id of the tile the machine returns to and the message that the machine returns.

```
[IMPORTANT] Farm Machine: Machine $machineID is finished and returns to the shed at $tileID.
```

In case a machine was not able to return, it remains on the tile. Then, it has to output to the output handle a log message containing the type of the log message (i.e., `Farm Machine`) with the `id` of the machine and the message that the machine failed to return. The machine makes no other attempt at return, it does not perform any actions in the remaining simulation.

```
[IMPORTANT] Farm Machine: Machine $machineID is finished but failed to return.
```

In case the machine had harvest loaded, it unloads at the shed it has returned to. Therefore, it has to output to the output handle a log message containing the type of the log message (i.e., `Farm Machine`) with the `id` of the machine and the amount and `plant` it harvested and unloads and the message that the machine unloads harvest.

```
[IMPORTANT] Farm Machine: Machine $machineID unloads $amount g of $plant harvest in the shed.
```

After the farm has performed all its actions, the farm has to output to the output handle a log message containing the type of the log message (i.e., `Farm`) with the `id` of the farm and the message that the farm finished its actions.

```
[IMPORTANT] Farm: Farm $farmID finished its actions.
```

Incidents After the farms have performed their actions, incidents can occur. The incidents happen at the end of the tick, in the tick specified inside the incident in order from the lowest `id` to the highest `id`. The incidents can be of different types, and the incidents can have different effects on the simulation. After the incident has happened, the incident has to output to the output handle a log message containing the type of the log message (i.e., `Incident`) with the `id` of the incident, `type` of the incident, and the message that the incident happened. Additionally, it also includes the `ids` of the tiles that were affected by the incident and the message that the tiles were affected.

The definition of affected tiles is as follows: For `CLOUD_CREATION` these are all tiles where new clouds are being created. For `CITY_EXPANSION`, it is the tile that is transformed into a `VILLAGE` tile. For `ANIMAL_ATTACK`, `BEE_HAPPY`, and `DROUGHT`, these are all tiles actually impacted by the incident or where the incident triggers a change in the computation of the harvest estimation. This counts even if the estimated harvest of a plant would already be $0\,\mathrm{g}$ prior to the effect of the incident. For `BROKEN_MACHINE`, it is the tile where the broken machine is located.

```
[IMPORTANT] Incident: Incident $incidentID of type $incidentType happened and affected tiles $tileIDs.
```

In case the incident is a CLOUD_CREATION incident and the newly created cloud merges with an existing one, the clouds need to log this union like the cloud union above. In that case, $cloudIDFromTile requires the id of the already existing cloud and $cloudIDMovingToTile of the newly created cloud.

Compute the Harvest Estimate After the farms have performed their actions and the incidents occurred, the harvest estimate must be recomputed for every FIELD and PLANTATION tile in order of increasing tile id. To indicate which actions were missed that influence the harvest estimate, the tile has to output to the output handle a log message containing the type of the log message (i.e., Harvest Estimate) with the id of the tile, the action that was missed, and the message that the estimated harvest changed. The actions are included in the following order: WEEDING, CUTTING, MOWING, IRRIGATING, HARVESTING.

```
[DEBUG] Harvest Estimate: Required actions on tile $tileID were not performed: $actions.
```

For every tile where the harvest has changed from the beginning of the tick until now, the tile has to output to the output handle a log message containing the type of the log message (i.e., Harvest Estimate) with the id of the tile, the new harvest estimate, the plant growing in the tile, and the message that the estimated harvest changed.

```
[INFO] Harvest Estimate: Harvest estimate on tile $tileID changed to $amount g of $plant.
```

Only after one tile has logged those logs as needed, does the next tile start logging.

2.3.4. Simulation Statistics

After the simulation has ended, your program has to output to the output handle a log message containing the type of the log message (i.e., Simulation Info) and the message that the simulation statistics are calculated.

```
[IMPORTANT] Simulation Info: Simulation statistics are calculated.
```

To calculate the simulation statistics, you have to calculate the amount harvested by each farm and the amount harvested for each plant, and the current estimate of the harvest still in the FIELD and PLANTATION tiles, respectively. Harvest counts as collected if it is on a machine or unloaded at a shed at the end of the simulation. Afterwards you have to output a log message for each farm (starting from the lowest id to the highest) containing the type of the log message (i.e., Simulation Statistics) with the id of the farm, the amount of harvest collected by the farm (across plants), and the message that the farm collected this amount of harvest.

```
[IMPORTANT] Simulation Statistics: Farm $farmID collected $amount g of harvest.
```

Next are the logs for each plant, in order of: `POTATO`, `WHEAT`, `OAT`, `PUMPKIN`, `APPLE`, `GRAPE`, `ALMOND`, `CHERRY`. You have to output to the output handle a log message containing the type of the log message (i.e., `Simulation Statistics`) with the plant and the amount of harvest collected.

```
[IMPORTANT] Simulation Statistics: Total amount of $plant harvested:
$amount g.
```

Finally, you have to output to the output handle a log message containing the type of the log message (i.e., `Simulation Statistics`) with the current estimate of all harvest remaining in the `FIELD` and `PLANTATION` tiles.

```
[IMPORTANT] Simulation Statistics: Total harvest estimate still in
fields and plantations: $amount g.
```

3. Technical Details

3.1. Breadth First Search algorithm

Breadth First Search (BFS)[1] is a popular algorithm to search in an unweighted graph structure (unweighted=every edge has equal relevance). In contrast to other search algorithms on graphs, it starts by visiting the closest nodes first and then draws ever wider circles to identify nodes adjoining to the ones already visited. Thus, it can be used to identify nodes in proximity to a starting point that have a specific property (e.g., specific tile category, ...). In our case, we are interested in identifying all reachable nodes or all nodes within a given distance that fulfill a given property. You can find a pseudocode description below. However, you might need to modify this algorithm a bit to stop and return the node when reaching a node with given property. It is especially important to notice that the algorithm might not be able to stop once it has found a suitable node, as we use different tie-break rules (e.g., lowest id) than the algorithm.

[1] You can find an explanation of the algorithm on wikipedia

```
"""
input:
* a graph (G),
* a starting node of G (startNode),
* a function that for each node in G either identifies the node as fulfilling
     the property or not (nodeProp),

output: a node fulfilling the property or None.
"""
fun BFS(graph:Graph, startNode:Node, nodeProp:(Node)->Boolean): Node or None
    {
  visited, toVisit = List<Node>(), List<Node>()
  visited.append(startNode)
  toVisit.append(startNode)
  while (toVisit not empty) {
    currentNode = toVisit.pop()
    if (nodeProp(currentNode))
       return currentNode
    else {
      for (adjNode in graph.adjoiningNodes(currentNode)) {
        if (adjNode in visited) continue
        visited.append(adjNode)
        toVisit.append(adjNode)
      }
    }
  }
  return None
}

```

Figure 8: Pseudocode for BFS

3.2. Command-Line Interface

The simulator is started with these command line parameters:

`--map` Path to the map. (always required) String

`--farms` Path to the file with information about the farms. (always required) String

`--scenario` Path to the scenario file. (always required) String

`--start_year_tick` The tick to start with within a year (optional, default value 1, between 1 and 24 inclusive) Int

`--max_ticks` Maximum allowed number of simulation ticks. (always required, must not exceed $1'000$) Int

`--log_level` The level of log detail that shall be output (always required, either `DEBUG`, `INFO` or `IMPORTANT`) String

`--out` Path to output file. Uses 'stdout' by default. String

`--help` Usage info

3.3. Build Script

We provide you with a build script. A build script is responsible for managing the dependencies of a software project and creating a finished executable program from the project files. As a build tool, we use *gradle*[2].

The build script can be found in the `build.gradle.kts` file in the *implementation* repository. Changes to your build script will be rolled back from the test server for execution. You can (and should) build the project yourself by either running the `./gradlew build` command or running the *gradle* task `build` directly in an IDE (e.g., *IntelliJ*). We use *Kotlin 2.1.21* with the *Java JDK Version 21* from *Eclipse Temurin 21.0.8*. You can download it either via the website[3] or directly in IntelliJ. In case you download it via IntelliJ, please make sure to set `JAVA_HOME` to the folder of the Java installation and add its `bin` folder to the beginning of the `PATH` variable.

In the build script there are dependencies on various libraries. We recommend that you look at these libraries and consider whether you can use them. Other libraries that are not entered in the build script must not be used. For security reasons, we do not allow reflection, network connections, or other connections to the outside world. Also, with the exception of loading configuration files and creating an output file, you may not read, create, or modify files in the file system. For loading configuration files and writing an output file, you may only access the paths given in the command line parameters and the *resources* folder for reading the JSON schema.

[2] https://gradle.org/
[3] https://adoptium.net/de/temurin/releases?version=21&os=any&arch=any

3.4. Code Quality

Good code quality helps software to remain readable and it helps to avoid errors. In the SE Lab, you will learn about automated tools that help you write good and (hopefully) bug-free code. Specifically, we use the tool DETEKT[4]. You can use this tool with *gradle*.

DETEKT is a static analysis tool, which means it doesn't need to run your code to find errors. DETEKT examines the source code for certain patterns that are known to cause errors. Examples of such patterns are unreachable code (you probably forgot to call a method) or using 200 arguments in a function. You will notice that DETEKT does not allow `print|println`. For the logging functionality, you have to use a `PrintWriter` from the package `java.io`.

Furthermore, DETEKT provides style rules in order to check your code for style issues (e.g., missing documentation for public functions). This helps improve readability and maintainability of your code.

You can find a report of the analysis in the directory `build/reports/detekt`.

[4] https://detekt.dev/docs/intro

4. Tests

In the group phase we expect system, unit, and integration tests from you. You can find the requirements, which are demanded from your application, in the specifications from the previous chapters.

Try to achieve the highest possible statement and branch coverage in the implementation phase. This means that in your code as many statements and branches as possible should be covered by the tests. The coverage can be calculated, e.g., with *JaCoCo*[1], also *IntelliJ* has already integrated tools for this.

4.1. Unit Tests

For unit tests, use *JUnit 5*[2] and *Mockito*[3] which is integrated into the project via the `Mockito-Kotlin` wrapper. Various tutorials for using these frameworks can be found on the Internet[4]. We will provide you with a sample test at the beginning of the implementation phase.

The unit tests can be run with `./gradlew test` (or `gradlew.bat test`).

Hint for implementation

Unit tests belong in the given directory `src/test/kotlin` in the project folder.

4.2. Integration Tests

It is your job to also write integration tests to test how the modules work together. In integration tests, you access individual methods similar to unit tests. You can write and execute these tests like Unit Tests.

[1] https://www.jacoco.org/
[2] https://junit.org
[3] https://site.mockito.org
[4] https://junit.org/junit5/docs/current/user-guide/

4.3. System Tests

You also need to write system tests that sufficiently test the simulator for all functionality.

In order to test the simulator, you have to provide the configuration files for the simulation scenario you want to test. Your system tests will also run on the reference implementation, so that you can determine if your assumption about the simulation behavior is correct.

Note that your system tests are also run on faulty simulations (mutants), so your tests must find faulty implementations.

Hint for implementation

System tests belong in the given directory `src/systemtest/kotlin` in the project folder.

A. Appendix

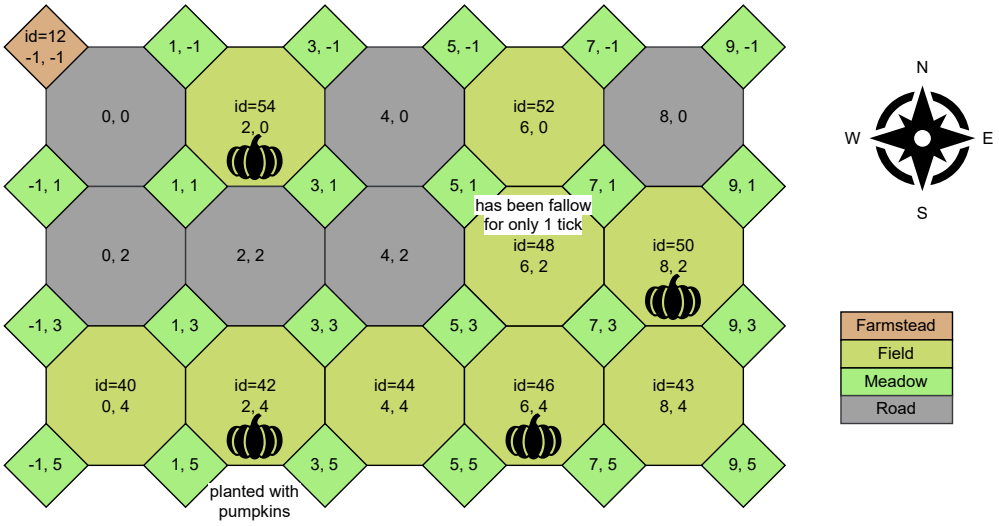


Figure 9: Example of a map, used for the 2nd sequence diagram

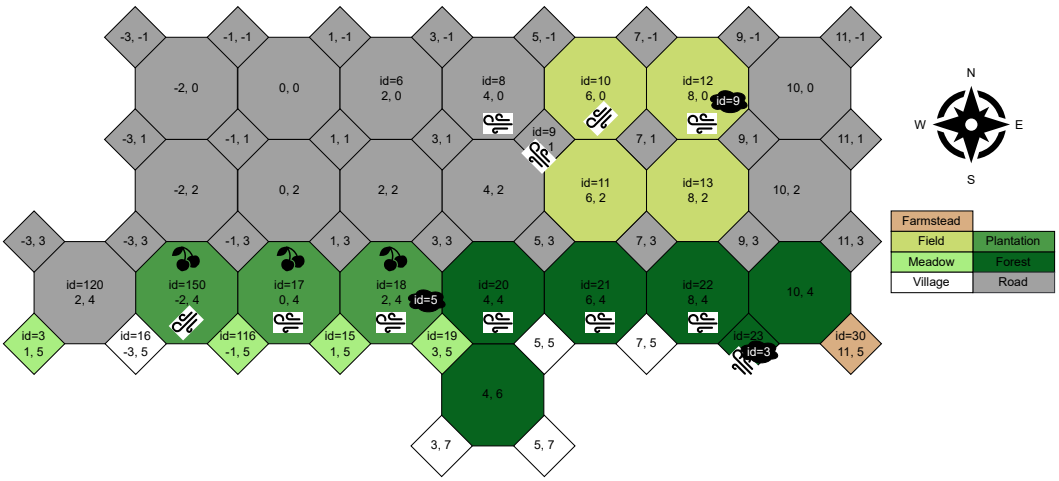


Figure 10: Example of a map, used for the 3rd sequence diagram